

Делегирование событий и деструктуризация

Распространение событий (event propagation)

Event propagation — термин, описывающий жизненный цикл события. Состоит из трёх фаз:

1. Погружение (capture phase)
2. Целевая фаза (target phase)
3. Всплытие (bubble phase)

На практике чаще всего используют только фазу всплытия.

- **Capturing phase (погружение)** — событие начинается на `window` и опускается через всех предков до самого глубокого целевого элемента.
- **Target phase (целевая фаза)** — событие достигло целевого элемента (просто сообщение, что действие произошло).
- **Bubbling phase (всплытие)** — событие поднимается от целевого элемента через всех предков обратно до `window`.

Распространение событий (event propagation) часто ошибочно считают синонимом всплытия — но всплытие лишь одна из фаз. Каждое событие проходит все три фазы.

Всплытие событий (event bubbling)

При наступлении события обработчики сначала срабатывают на самом вложенном элементе, затем на его родителе, и так далее вверх по цепочке предков до `window`. Это называется всплытием (bubbling) — по аналогии с пузырьком воздуха, поднимающимся в воде.

```
<div id="parent">
  Parent
  <div id="child">
    Child
    <div id="descendant">Descendant</div>
  </div>
</div>
```

Клик по `#descendant` вызовет обработчики в порядке: сначала **descendant**, затем **child**, затем **parent** — и далее вверх до `window`.

Целевой элемент

Где бы обработчик ни «поймал» событие во время всплытия — всегда можно узнать, где именно оно произошло.

- `event.target` — ссылка на элемент, где событие фактически произошло (исходный/целевой элемент). Остаётся неизменной на всех этапах всплытия.
- `event.currentTarget` — ссылка на элемент, к которому привязан текущий обработчик (тот, до которого дошло всплытие).

Если обработчик зарегистрирован на самом верхнем элементе, он благодаря всплытию перехватит клики со всех дочерних элементов. Важно различать: `event.target` — где событие произошло, `event.currentTarget` — где сработал обработчик.

Прекращение всплытия

Обычно событие всплывает до `window`, вызывая по пути все обработчики. Один элемент может иметь несколько обработчиков на одно событие.

Остановить всплытие можно методами объекта события:

Метод	Что делает
<code>event.stopPropagation()</code>	Останавливает всплытие в DOM-дереве — родительские элементы событие не получают. Другие обработчики на том же элементе продолжают выполняться.
<code>event.stopImmediatePropagation()</code>	Останавливает всплытие так же, как <code>stopPropagation()</code> , и дополнительно отменяет выполнение всех остальных обработчиков этого события на том же элементе.

Не останавливайте всплытие без необходимости. Это создаёт побочные проблемы: например, веб-аналитика использует всплытие для отслеживания действий пользователя — остановка всплытия сделает сбор этих данных невозможным.

Делегирование событий (event delegation)

Всплытие позволяет реализовать один из самых полезных приёмов — делегирование событий (event delegation): вместо обработчика на каждом элементе группы вешаем один обработчик на их общего предка (common ancestor).

Пример: `<div>` со 100 кнопками внутри. Вместо 100 отдельных слушателей — один слушатель на `<div>`, который ловит клики благодаря всплытию.

Делегирование в три шага:

1. Найти общего предка группы элементов.
2. Зарегистрировать на предке обработчик нужного события.
3. В обработчике использовать `event.target`, чтобы определить конкретный элемент, на котором событие произошло.

```
const box = document.querySelector(".box");

box.addEventListener("click", function (event) {
  console.log(event.target); // элемент, на котором произошёл click
});
```

Такой подход упрощает работу с однотипными элементами: их можно добавлять, удалять, менять — без ручного управления обработчиками на каждом.

Проверка целевого элемента события

Пример: палитра цветов (color palette). Клик по элементу палитры меняет цвет и текст. Вместо обработчика на каждой кнопке — один слушатель на общем предке `div.color-palette`.

```
const colorPalette = document.querySelector(".color-palette");

colorPalette.addEventListener("click", selectColor);

function selectColor(event) {
  console.log(event.target);
  const selectedColor = event.target.dataset.color;
}
```

Цвет хранится в атрибуте `data-color`. Обязательно нужно проверять тип целевого элемента через `nodeName` — иначе клик между кнопками может вызвать ошибку.

```
function selectColor(event) {  
  if (event.target.nodeName !== "BUTTON") {  
    return; // клик пришёлся между кнопками  
  }  
  
  const selectedColor = event.target.dataset.color; // клик точно по кнопке  
}
```

Библиотеки (Libraries)

Что такое библиотека

Библиотека (library) — набор готовых функций, методов и классов, которые дают разработчику инструменты для решения типовых задач без написания кода с нуля.

- **Многократное использование (reusability)** — можно подключать в разные проекты.
- **Готовый функционал** — под типовые задачи (даты, строки, анимации и т.д.).
- **Ускорение разработки** — не нужно писать и отлаживать код заново.
- **Поддержка сообществом (community support)** — обновления, исправления багов, обратная совместимость.

Примеры: **Chart.js** (графики и диаграммы), **Lodash** (работа с массивами, объектами, строками).

CDN (Content Delivery Network)

CDN — географически распределённая сеть серверов, которая доставляет контент (стили, скрипты, изображения) с сервера, ближайшего к пользователю, — это ускоряет загрузку.

Преимущество	Суть
Быстрая загрузка	Контент грузится с географически близкого сервера
Распределённая нагрузка	Серверы CDN разбросаны по миру
Доступность	Если один сервер недоступен — сработает другой
Снижение нагрузки на источник	Основной сервер проекта не перегружается

Подключение библиотеки через CDN

На примере Lodash через сервис **jsdelivr.com**:

1. Найти библиотеку на CDN-сервисе по имени.
2. Скопировать готовый тег `<script>` со страницы библиотеки.
3. Вставить его в HTML — **перед** подключением собственного скрипта.

```
<body>  
  <!-- скрипт библиотеки Lodash -->  
  <script async src="https://cdn.jsdelivr.net/npm/lodash@4.17.21/lodash.min.js"></script>  
  <!-- свой файл скрипта -->  
  <script defer src="path/to/script.js"></script>  
</body>
```

Тег `<script>` библиотеки подключается раньше своего скрипта и обязательно с атрибутом `async` — чтобы файл библиотеки загружался как можно быстрее.

При подключении через CDN библиотека добавляет своё свойство в объект `window` — имя свойства уникально для каждой библиотеки и указано в её документации. У Lodash это символ подчёркивания `_`.

```
console.log(_.sum([4, 2, 8, 6])); // 20
console.log(_.shuffle([1, 2, 3, 4])); // перемешанный массив
```

Как читать документацию библиотеки

Дизайн документации у каждой библиотеки свой, но обязательно есть инструкции по установке, примеры использования и описание настроек. Названия разделов обычно стандартны:

Что искать	Типичные названия разделов
Установка и подключение	Get Started, Quick Start, Installation, Setup
Примеры использования	Usage, Examples, API, Demo
Настройки и особенности	Options, Advanced Settings, Configuration

На примере библиотеки **basicLightbox**: метод `create()` создаёт экземпляр (instance) — принимает контент (обязательный параметр) и объект настроек (необязательный). У экземпляра есть свои методы, например `show()` и `close()`.

```
const instance = basicLightbox.create(`

# 


```

Пример опций: `className` — добавляет свои классы для стилизации, `onShow` — функция, вызываемая при открытии окна.

Деструктуризация (Destructuring)

Зачем нужна деструктуризация

Деструктуризация (destructuring) — синтаксис, который позволяет извлечь значения из объектов или массивов и сразу присвоить их переменным, не обращаясь к каждому свойству через точку.

```
// Без деструктуризации
const user = { name: "Jacob", age: 32 };
console.log(user.name); // Jacob

// С деструктуризацией
const { name, age } = user;
console.log(name); // Jacob
```

Деструктуризация объектов (object destructuring)

Синтаксис: после `const / let` ставятся фигурные скобки с именами переменных — так же, как при объявлении объекта.

- Деструктуризация всегда находится слева от знака присваивания.
- Переменная получает значение одноимённого свойства объекта.
- Порядок переменных внутри `{ }` не важен.

```
const book = {
  title: "The Last Kingdom",
  author: "Bernard Cornwell",
  isPublic: true,
};

const { title, author, isPublic } = book;
const accessType = isPublic ? "public" : "private";
```

Деструктуризация несуществующих свойств

Если в объекте нет свойства с таким именем — переменной присвоится `undefined`.

Чтобы избежать `undefined`, можно задать значение по умолчанию (default value) через `=`. Оно применится только при отсутствии свойства в объекте.

```
const book = { title: "The Last Kingdom" };

const {
  title,
  coverImage = "https://via.placeholder.com/640/480"
} = book;

console.log(coverImage); // плейсхолдер, т.к. свойства нет в объекте
```

Переименование переменной (renaming)

При деструктуризации можно задать переменной другое имя через двоеточие `:` — сначала имя свойства объекта, затем новое имя переменной.

```
const book = {
  author: "Bernard Cornwell",
  rating: 8.38,
};

const { author: bookAuthor, rating: bookRating } = book;
console.log(bookAuthor); // "Bernard Cornwell"
console.log(bookRating); // 8.38
```

Значение по умолчанию при переименовании

При переименовании переменной также можно задать значение по умолчанию — оно ставится после нового имени через `=`. Если свойство есть в объекте — берётся его значение, если нет — значение по умолчанию.

```
const book = { title: "The Dream of a Ridiculous Man" };

const {
  title,
  coverImage: bookCoverImage = "https://via.placeholder.com/640/480",
} = book;

console.log(bookCoverImage); // плейсхолдер — свойства coverImage нет в объекте
```

Деструктуризация в циклах (destructuring in loops)

При переборе массива объектов через `for...of` деструктуризация избавляет от повторных обращений через точку. Можно деструктурировать в теле цикла или сразу в объявлении переменной цикла.

```
for (const { title, author, rating } of books) {  
  console.log(title, author, rating);  
}
```

Деструктуризация параметров функции (destructuring parameters)

Позволяет получать нужные свойства объекта прямо в параметрах функции, без обращения через точку в теле функции.

```
function printUserInfo({ name, age, hobby }) {  
  console.log(`Name: ${name}, Age: ${age}, Hobby: ${hobby}`);  
}  
  
printUserInfo({ name: "Alice", age: 25, hobby: "dancing" });
```

Паттерн «Объект параметров» (parameter object pattern)

Если функция принимает 3 и больше параметров — легко запутаться, что и в каком порядке передавать при вызове.

```
// ❌ непонятно, что означают 736, 10283, true  
doStuffWithBook("The Last Kingdom", 736, 10283, 8.38, true);
```

Решение — заменить набор параметров одним объектом с именованными свойствами, и сразу деструктурировать его в сигнатуре функции:

```
// ✅ понятно, что есть что  
function doStuffWithBook({ title, pages, downloads, rating, isPublic }) {  
  console.log(title, pages);  
}  
  
doStuffWithBook({  
  title: "The Last Kingdom",  
  pages: 736,  
  downloads: 10283,  
  rating: 8.38,  
  isPublic: true,  
});
```

Глубокая деструктуризация (deep / nested destructuring)

Для вложенных объектов принцип тот же: после имени свойства ставим `:` и открываем деструктуризацию вложенного объекта. Можно сочетать с переименованием и значениями по умолчанию.

```
const user = {  
  name: "Jacques Gluke",  
  stats: { followers: 5603, views: 4827, likes: 1308 },  
};  
  
const {  
  name,  
  stats: { followers = 0, views: userViews = 0 },  
} = user;  
  
console.log(name, followers, userViews); // Jacques Gluke 5603 4827
```

Деструктуризация массивов (array destructuring)

Синтаксис

Работает по тем же принципам, что и деструктуризация объектов, но с отличиями:

- Вместо фигурных скобок `{}` используются квадратные `[]`.
- Значения присваиваются переменным **по порядку** (по индексу элемента), а не по имени.

```
const color = [200, 255, 100];
const [red, green, blue] = color;

console.log(`rgb(${red}, ${green}, ${blue})`); // "rgb(200, 255, 100)"
```

Значения по умолчанию

Если переменных объявлено больше, чем элементов в массиве — лишним присвоится `undefined`. Чтобы этого избежать, задаём значение по умолчанию через `=`, как и для объектов.

```
const color = [200, 100, 255];
const [red, green, blue, alfa = 0.3] = color;

console.log(`rgba(${red}, ${green}, ${blue}, ${alfa})`); // "rgba(200, 100, 255, 0.3)"
```

Частичная деструктуризация (rest в деструктуризации)

Можно распаковать только первые нужные элементы массива, а остальные собрать в новый массив с помощью `...rest`. Оригинальный массив при этом не меняется.

```
const color = [200, 255, 100];
const [red, ...otherColors] = color;

console.log(red); // 200
console.log(otherColors); // [255, 100]
```

Работает и с объектами: часть свойств выносим в переменные, остальные собираются в новый объект.

```
const user = { name: "Jacob", age: 32, email: "j.cob@mail.com", isOnline: true };

const { name, isOnline, ...otherProps } = user;

console.log(name, isOnline); // "Jacob" true
console.log(otherProps); // { age: 32, email: "j.cob@mail.com" }
```

Пропуск значений (skipping elements)

Элементы массива — это индексированные значения (в отличие от именованных свойств объекта). Чтобы деструктурировать, например, только третий элемент, первые два нужно пропустить, оставив запятые пустыми.

```
const rgb = [200, 100, 255];
const [, , blue] = rgb;

console.log(`Blue: ${blue}`); // "Blue: 255"
```

На практике для массивов приём используется редко, но полезно знать, что так можно.

Деструктуризация параметров-массивов

Массив, переданный в функцию, тоже можно деструктурировать прямо в параметрах.

```
function printFruits([firstFruit, secondFruit, thirdFruit]) {  
  console.log(firstFruit, secondFruit, thirdFruit);  
}  
  
printFruits(["apple", "banana", "orange"]); // apple banana orange
```

Преимущества деструктуризации

- **Удобное извлечение значений** — не нужно каждый раз обращаться к полям объекта или элементам массива по имени/индексу.
- **Короткий и читаемый код** — вместо длинных обращений сразу получаем переменные с понятными именами.
- **Параметры функций** — явно видно, какие поля объекта использует функция.
- **Работа с возвращаемыми объектами** — результат функции можно сразу деструктурировать, не сохраняя его целиком в переменную.